

Events & Pub-Sub

The node's nervous system — an in-process publish/subscribe bus for components to react to each other, and a durable, replayable event queue for the outside world.

SUBJECT hathor-core · Part II · the node, end to end

CHAPTER 30 · Part II · The Node

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Pub-sub · Observer · PubSubManager · HathorEvents · EventManager · Event queue ·
Replayable feed · Decoupling · WebSocket streaming

Table of Contents

Chapter 30 — Events & Pub-Sub	3
30.1 Localization	3
30.2 What it does and why it exists	4
30.3 The concepts it rests on	6
30.4 The code, walked — Layer 1: the in-process bus	7
A tiny pub-sub toy first	7
The catalogue: HathorEvents	7
The argument bag: EventArguments	8
PubSubManager: subscribe and publish	8
30.5 The code, walked — Layer 2: the durable event queue	10
It is, itself, a subscriber	10
Translating a bus event into a durable record	11
The persisted record: BaseEvent	12
Receive → persist → broadcast	12
The storage: EventRocksDBStorage	13
Streaming to the world: the WebSocket layer	13
The replayable backlog at startup	14
30.6 How it plugs into the lifecycle	14
Recap	16

Chapter 30 — Events & Pub-Sub

What you'll learn in this chapter

- Why a node needs an announcement system at all — and the cost of wiring components together without one.
- The **two layers** Hathor builds, and why they are separate: the in-process **PubSubManager** (ephemeral, observer-pattern, internal) versus the durable **EventManager** and its **event queue** (persisted, replayable, external-facing).
- How **PubSubManager** works in code: the **HathorEvents** catalogue, **subscribe / publish**, and the subtle detail of when a published callback actually runs (it usually waits for the reactor).
- How **EventManager** subscribes to the bus, turns ephemeral events into numbered, persisted **BaseEvent** records in RocksDB, and streams them to external clients over WebSocket — with acknowledgement and flow control.
- Where both layers are created and started in the node's lifecycle, and who fires the events you'll most often care about.

Chapter 29 introduced the **HathorManager** — the coordinator that holds every subsystem and starts them in order. This chapter is about how those subsystems talk to each other after startup, without being wired together by hand. It is the payoff for two earlier chapters: the **observer / publish-subscribe** pattern from Chapter 3, and the **callback** mechanics from Chapter 2. If those are hazy, the recap boxes below will refresh them in context; the full treatments live where named.

30.1 Localization

The event machinery lives in two places. The first is a single file at the top of the **hathor/** package; the second is a sub-package next to it.

hathor-core/	
└─ hathor/	
pubsub.py	← the in-process bus ◀ YOU ARE HERE (Layer 1)
manager.py	← creates pubsub, starts event_manager (Ch 29)
vertex_handler/	← fires NEW_TX_ACCEPTED & friends (Ch 33)
└─ event/	← the durable, external feed ◀ (Layer 2)
__init__.py	
event_manager.py	← EventManager: subscribes to pubsub, persists, broadcasts; the heart of Layer 2
└─ model/	
base_event.py	← BaseEvent: the persisted, numbered record
event_type.py	← EventType enum + map from HathorEvents
event_data.py	← typed payloads (TxData, ReorgData, ...)
node_state.py	← LOAD / SYNC node-state marker
└─ storage/	
event_storage.py	← EventStorage ABC (the interface)
rocksdb_storage.py	← EventRocksDBStorage (the real backend)
└─ websocket/	
factory.py	← EventWebSocketFactory: holds connections, broadcasts events
protocol.py	← EventWebSocketProtocol: per-client state machine
request.py	← client → node messages (START_STREAM, ACK, ...)
response.py	← node → client messages (EventResponse, ...)
└─ resources/	
event.py	← HTTP endpoint to read events (not streaming)

Context. pubsub.py is the node's internal nervous system: any component can announce "something happened" and any other component can listen, with neither knowing about the other. The event/ package builds a second system on top of that bus — a durable, ordered, replayable feed that the outside world (dashboards, exchanges, indexers) can subscribe to over a WebSocket and never miss a beat, even across restarts. The first is for the node talking to itself; the second is for the node talking to the world.

30.2 What it does and why it exists

Start with the problem, because the design only makes sense against it.

A running node is full of moments worth announcing. A new transaction gets accepted. A peer connects, then becomes ready, then disconnects. The consensus algorithm reorganizes the chain. A nano-contract finishes executing. Each of these moments matters to several parts of the program at once. When a transaction is accepted, the address index wants to record which addresses it touched, the metrics counter wants to tick up, the wallet may want to update a balance, and any connected WebSocket dashboard wants to be told.

The naive way to handle this is direct wiring. Inside the code that accepts a transaction, you would write:

```
self.address_index.add_tx(tx)
self.metrics.tx_accepted += 1
self.wallet.on_new_tx(tx)
self.websocket.notify(tx)
```

This works, and for a two-component program it is fine. But it has a structural flaw that grows with the codebase. The transaction-acceptance code now knows about the address index, the metrics object, the wallet, and the WebSocket server. It imports them, holds references to them, and must be edited every time a new listener appears or an old one is removed. The announcer is welded to its audience. In a node with dozens of subsystems, this welding becomes a web of cross-references that is hard to reason about and harder to change.

Recap — observer / publish-subscribe (full treatment in Ch. 3, §3.8). The pattern inverts the wiring. The announcer — the publisher — keeps a list of interested parties (subscribers) and notifies all of them when an event occurs, knowing nothing about who they are or what they do. Subscribers register a callback to be run on each event. The result is **decoupling**¹: you can add a subscriber without touching the publisher, and remove one without anyone noticing. The transaction-acceptance code shrinks to one line — `publish("new tx accepted", tx=tx)` — and stops caring who listens.

So the first thing the event system does is **decouple producers of events from consumers of events**. That is the job of `PubSubManager` (Layer 1). It is in-process, in-memory, and ephemeral: an event is published, every current subscriber's callback runs, and then the event is gone. Nobody who was not subscribed at that instant will ever see it. This is exactly right for components inside the same process that are all alive at the same time.

But there is a second, different need. Systems outside the node — a block explorer, a cryptocurrency exchange, an analytics pipeline — want a complete, ordered, gap-free record of everything the node has seen, and they want to be able to disconnect, reconnect, and pick up exactly where they left off. An ephemeral in-memory bus cannot serve them: if the consumer is offline when an event fires, the event is lost forever, and there is no notion of "event number 4,217, please resume from there."

That is why Layer 2 exists. The `EventManager` subscribes to the in-process bus, and for the events that matter to outsiders it does three things the bus does not: it assigns each event a **sequential id**, it **persists** the event to RocksDB, and it **broadcasts** it to connected WebSocket clients with acknowledgement and flow control. The persisted, numbered log is the **event queue**²: a durable, replayable feed. A consumer can ask "send me everything from id 4,217 onward," go offline, come back, and resume — because the events are on disk, not in volatile memory.

The two layers, then, answer two different questions:

	Layer 1 — <code>PubSubManager</code>	Layer 2 — <code>EventManager</code> + queue
Audience	the node's own components	external systems
Lifetime	one notification, then gone	persisted forever (until pruned)
Ordering	none guaranteed	strict, gap-free <code>id</code> sequence
Replay	impossible	the whole point
Built on	nothing (it is the base)	the Layer-1 bus

Hold that distinction; the rest of the chapter fills in each layer.

30.3 The concepts it rests on

Before the code, three quick recaps. None is re-taught from scratch here — each points to its canonical chapter.

Recap — callbacks (full treatment in Ch. 2). A callback is a function you hand to someone else to be called later, when some condition is met. Pub-sub is callbacks at scale: subscribing is "here is a function, call it when this event happens." Everything in this chapter is ultimately a list of callbacks being invoked.

Recap — the reactor and `callLater` (full treatment in Ch. 16, recapped in Ch. 23). Hathor runs on Twisted's **reactor**, a single event loop that waits for things to happen and dispatches handlers. `reactor.callLater(0, fn)` schedules `fn` to run "as soon as the loop is free" rather than right now. The pub-sub bus uses this to avoid running subscriber callbacks in the middle of whatever code just published — a detail we examine in §30.4.

Recap — RocksDB persistence (full treatment in Ch. 27). RocksDB is the embedded key-value store the node uses for everything on disk. It organizes data into named **column families** — independent key spaces inside one database. The durable event queue is just two more column families (one for events, one for metadata) in the same RocksDB instance that holds the ledger. → full treatment in Ch. 27.

With those in hand, we walk the code.

30.4 The code, walked — Layer 1: the in-process bus

A tiny pub-sub toy first

The whole of `PubSubManager` is an elaboration of one twelve-line idea. Here it is in neutral Python, the same shape you met in Chapter 3:

```
class Bus:
    def __init__(self):
        self._subscribers = {}          # event name → list of callbacks

    def subscribe(self, event, callback):
        self._subscribers.setdefault(event, []).append(callback)

    def publish(self, event, **data):
        for callback in self._subscribers.get(event, []):
            callback(event, data)

bus = Bus()
bus.subscribe("new_tx", lambda ev, d: print("index sees", d["tx"]))
bus.publish("new_tx", tx="abc123")      # → index sees abc123
```

A dictionary from event name to a list of callbacks; `subscribe` appends; `publish` loops and calls. Hold this picture — the real class is this plus three refinements: a typed catalogue of event names, a typed argument bag, and a reactor-aware delivery mechanism.

The catalogue: `HathorEvents`

The real bus does not key on free-form strings. It keys on an enum so that a typo cannot silently create a new, unlistened-to event. The catalogue lives at `hathor/pubsub.py:37`:

```
class HathorEvents(Enum):
    MANAGER_ON_START = 'manager:on_start'
    MANAGER_ON_STOP = 'manager:on_stop'
    NETWORK_PEER_CONNECTED = 'network:peer_connected'
    NETWORK_NEW_TX_PROCESSING = 'network:new_tx_processing'
    NETWORK_NEW_TX_ACCEPTED = 'network:new_tx_accepted'
    CONSENSUS_TX_UPDATE = 'consensus:tx_update'
    CONSENSUS_TX_REMOVED = 'consensus:tx_removed'
    REORG_STARTED = 'reorg:started'
    REORG_FINISHED = 'reorg:finished'
    # ... wallet, peer, nano-contract events ...
```

The enum members are grouped by namespace in their string values (`manager:`, `network:`, `consensus:`, `reorg:`, `wallet:`, `nc:`), which is documentation for the human reader; the code only ever compares enum members. The class's docstring (`pubsub.py:38`) describes, per event, when it fires and what it publishes — read it as the authoritative index of the node's internal life. The two you will see most are `NETWORK_NEW_TX_ACCEPTED` (`pubsub.py:127`) — fired whenever a vertex is accepted into the ledger — and `MANAGER_ON_START` (`pubsub.py:112`) — fired once when the node boots.

The argument bag: `EventArguments`

Different events carry different data. A transaction-accepted event carries a `tx`; a reorg event carries the old and new best blocks and the reorg size. Rather than force every callback to accept a fixed signature, the bus wraps the published keyword arguments in a small object, `EventArguments` (`pubsub.py:160`):

```
class EventArguments:
    def __init__(self, **kwargs: Any) -> None:
        for key, value in kwargs.items():
            setattr(self, key, value)

    def __contains__(self, key: str) -> bool:
        return hasattr(self, key)
```

It does nothing but turn `EventArguments(tx=vertex)` into an object with a `.tx` attribute, and let `'tx' in args` test for presence. The class body lists the attributes that may appear, with their types (`pubsub.py:164`), as a hint to readers and type-checkers — but any given instance only has the attributes that were actually passed. A subscriber reads the fields it knows its event carries.

The callback signature is fixed by a type alias at `pubsub.py:185`:

```
PubSubCallable = Callable[[HathorEvents, EventArguments], None]
```

Every subscriber is a function taking the event key and the argument bag, returning nothing.

PubSubManager : subscribe and publish

The bus itself is `PubSubManager` (`pubsub.py:188`). Its state (`pubsub.py:194`) is the dictionary you'd expect, plus a delivery queue and a reactor reference:

```
def __init__(self, reactor: Reactor) -> None:
    self._subscribers = defaultdict(list)
    self.queue: deque[tuple[PubSubCallable, HathorEvents, EventArguments]] = deque()
    self.reactor = reactor
    self.log = logger.new()
    self._call_later_id: IDelayedCall | None = None
```

`subscribe` (`pubsub.py:204`) is the toy's `subscribe` with one guard — a callback is not added twice:

```
def subscribe(self, key: HathorEvents, fn: PubSubCallable) -> None:
    if fn not in self._subscribers[key]:
        self._subscribers[key].append(fn)
```

`unsubscribe` (`pubsub.py:216`) removes it. So far, the toy.

The interesting part is `publish` (`pubsub.py:257`), because of when it runs the callbacks:

```
def publish(self, key: HathorEvents, **kwargs: Any) -> None:
    args = EventArguments(**kwargs)
    for fn in self._subscribers[key]:
        if not self.reactor.running:
            fn(key, args)
        else:
            self.queue.append((fn, key, args))
            self._schedule_call_next()
```

Read this carefully — it is the one genuinely non-obvious thing in the whole bus. There are two delivery modes:

1. **Reactor not running** (startup, or tests). Subscribers are called **synchronously, right now**, in the publishing call's own stack. There is no event loop yet to defer to.
2. **Reactor running** (normal operation). Each `(callback, key, args)` triple is **appended to a queue**, and a single deferred drain is scheduled via the reactor.

Why defer during normal operation? Because publishing often happens deep inside critical code — for instance, in the middle of accepting a vertex, while storage locks may be held and metadata is mid-update. Running an arbitrary subscriber's callback synchronously at that moment risks re-entrancy: the callback might publish another event, or call back into the half-updated subsystem. By queueing the callbacks and running them on the next reactor tick, the bus guarantees that subscribers run after the publishing operation has fully returned and the stack has unwound — a clean, well-defined point.

The drain is `_call_next` (`pubsub.py:222`):

```
def _call_next(self) -> None:
    if not self.queue:
        return
    try:
        while self.queue:
            fn, key, args = self.queue.popleft()
            fn(key, args)
    except Exception:
        self.log.error('event processing failed', key=key, args=args)
        raise
    finally:
        self._schedule_call_next()
```

It pops and calls every queued callback in FIFO order, then re-schedules itself in case new events were queued while it ran. Scheduling is `_schedule_call_next` (`pubsub.py:239`); its core is the reactor call at `pubsub.py:255`:

```
self._call_later_id = self.reactor.callLater(0, self._call_next)
```

`callLater(0, ...)` means "run this as soon as the loop is free." The method also guards against scheduling twice (`pubsub.py:252`) and, if called from a non-reactor thread, hops back onto the reactor thread via `callFromThread` (`pubsub.py:246`) — because Twisted requires that reactor-facing work happen on the reactor's own thread. That thread-safety hop is the reason `publish` can be called from a worker thread (e.g. the proof-of-work pool from Ch. 16) and still deliver safely.

The Layer-1 invariant. When the reactor is running, a published event is not delivered inline; it is delivered on a subsequent reactor tick, in `publish` order. Code that publishes must not assume subscribers have run by the time `publish` returns.

That is the entire in-process bus: a typed catalogue, a typed argument bag, a dictionary of callback lists, and a reactor-deferred drain. Every internal "X happened" announcement in the node flows through this one object. Among its many publishers, the most central is the **vertex handler** (Ch. 33), which on accepting a vertex publishes — in order — `NETWORK_NEW_TX_PROCESSING`, then any consensus-generated events, then `NETWORK_NEW_TX_ACCEPTED` (`hathor/vertex_handler/vertex_handler.py:259 – 262`).

30.5 The code, walked — Layer 2: the durable event queue

Everything so far is ephemeral. Now we build the persistent, external-facing feed on top of it. The central class is `EventManager` (`hathor/event/event_manager.py:59`), whose own docstring states the pipeline plainly (`event_manager.py:60`): "Events are received from PubSub, persisted on the storage and sent to WebSocket clients." Three stages — **receive, persist, broadcast** — map onto three collaborators: the `PubSubManager` it subscribes to, the `EventStorage` it writes to, and the `EventWebSocketFactory` it broadcasts through.

It is, itself, a subscriber

`EventManager` does not invent its own way of hearing about node activity. It is a Layer-1 subscriber — the bridge between the two layers. At startup it registers one handler for a fixed list of bus events (`event_manager.py:48`):

```
_SUBSCRIBE_EVENTS = [  
    HathorEvents.NETWORK_NEW_TX_ACCEPTED,  
    HathorEvents.REORG_STARTED,  
    HathorEvents.REORG_FINISHED,  
    HathorEvents.CONSENSUS_TX_UPDATE,  
    HathorEvents.CONSENSUS_TX_REMOVED,  
    HathorEvents.NC_EVENT,  
    HathorEvents.NC_EXEC_SUCCESS,  
]
```

and `_subscribe_events` (`event_manager.py:141`) wires them all to a single method:

```
def _subscribe_events(self) -> None:
    for event in _SUBSCRIBE_EVENTS:
        self._pubsub.subscribe(event, self._handle_hathor_event)
```

Note which bus events are not in the list: the peer-connection and wallet events stay purely internal. Only a curated subset of node activity is promoted to the durable, external feed — the events an outside indexer would actually want.

Translating a bus event into a durable record

When the bus delivers one of those events, `_handle_hathor_event` (`event_manager.py:176`) translates it from the internal vocabulary (`HathorEvents`) into the external one (`EventType`):

```
def _handle_hathor_event(self, hathor_event: HathorEvents, event_args: EventArguments) -> None:
    event_type = EventType.from_hathor_event(hathor_event)
    if event_type is not None:
        self._handle_event(event_type, event_args)
    ...
```

`EventType` (`hathor/event/model/event_type.py:31`) is a separate enum from `HathorEvents`, and the separation is deliberate: it is the **public** event vocabulary, and it does not match the internal one one-for-one. The mapping lives in a dictionary at `event_type.py:52`:

```
_HATHOR_EVENT_TO_EVENT_TYPE = {
    HathorEvents.NETWORK_NEW_TX_ACCEPTED: EventType.NEW_VERTEX_ACCEPTED,
    HathorEvents.REORG_STARTED:           EventType.REORG_STARTED,
    HathorEvents.REORG_FINISHED:          EventType.REORG_FINISHED,
    HathorEvents.CONSENSUS_TX_UPDATE:     EventType.VERTEX_METADATA_CHANGED,
    HathorEvents.CONSENSUS_TX_REMOVED:    EventType.VERTEX_REMOVED,
    HathorEvents.NC_EVENT:                 EventType.NC_EVENT,
}
```

Notice the renaming: the internal `NETWORK_NEW_TX_ACCEPTED` becomes the public `NEW_VERTEX_ACCEPTED`; `CONSENSUS_TX_UPDATE` becomes `VERTEX_METADATA_CHANGED`. The public names are chosen for the external reader, who thinks in terms of "a vertex was accepted," not the node's internal plumbing. Keeping the two enums separate means the internal events can be renamed or reorganized without breaking the contract external consumers depend on. The `EventType` enum also includes types that have no `HathorEvents` source — `LOAD_STARTED`, `LOAD_FINISHED`, `FULL_NODE_CRASHED`, `TOKEN_CREATED` (`event_type.py:32 - 41`) — which the `EventManager` emits directly, not via the bus.

The persisted record: `BaseEvent`

The durable record is `BaseEvent` (`hathor/event/model/base_event.py:25`), a Pydantic model³. Its fields are the contract with the outside world:

```
class BaseEvent(BaseModel):
    id: NonNegativeInt          # unique, sequential – determines event order
    timestamp: float            # informative only; may not be monotonic
    type: EventType             # one of the public event types
    data: EventData             # typed payload, varies by type
    group_id: Optional[NonNegativeInt] = None # links related events (e.g. a reorg)
```

Each field models something the consumer needs. The `id` (`base_event.py:29`) is the spine of the whole queue: it is a gap-free, monotonically increasing integer, and it is what makes replay possible — "resume from id N" is a meaningful request precisely because ids are sequential and stored. The comment at `base_event.py:30` is candid that `timestamp` is not reliable for ordering (the system clock can move backwards); ordering is `id`'s job, not the timestamp's. The `data` field is a typed payload whose concrete type depends on `type` — a mapping declared at `event_type.py:61` (`NEW_VERTEX_ACCEPTED` → `TxData` , `REORG_STARTED` → `ReorgData` , and so on) and enforced by a model validator (`base_event.py:62`) that rejects any event whose data shape does not match its type. The `group_id` (`base_event.py:38`) ties together a burst of related events — every `VERTEX_METADATA_CHANGED` emitted during one reorg shares the reorg's group id, so a consumer can treat the burst as one logical operation.

Receive → persist → broadcast

The translation method funnels into `_handle_event` (`event_manager.py:188`), which is the three-stage pipeline in four lines:

```
def _handle_event(self, event_type: EventType, event_args: EventArguments) -> None:
    assert self.is_running, 'Cannot handle event, EventManager is not started.'
    event = self._handle_event_creation(event_type, event_args) # number & build it
    self._event_storage.save_event(event)                      # persist (Layer 2 durability)
    self._event_ws_factory.broadcast_event(event)              # push to live clients
    self._last_event = event
```

`_handle_event_creation` (`event_manager.py:200`) assigns the next id and the right `group_id`, then builds the `BaseEvent`. The id assignment is at `event_manager.py:315` — 0 for the very first event, otherwise `self._last_event.id + 1` — which is what guarantees the gap-free sequence. The `group_id` is handled by three small helpers depending on whether the event opens a group (`REORG_STARTED` , `_GROUP_START_EVENTS` at `event_manager.py:40`), closes one (`REORG_FINISHED` , `_GROUP_END_EVENTS` at `event_manager.py:44`), or falls inside one.

Then the two effects that distinguish Layer 2 from Layer 1: **save to storage** (durable, replayable) and **broadcast to WebSocket clients** (live delivery). A Layer-1 publish does neither; it just calls callbacks. This is the concrete difference between the two layers, in two method calls.

The storage: `EventRocksDBStorage`

Persistence is an interface, `EventStorage` (`hathor/event/storage/event_storage.py`), with one real implementation, `EventRocksDBStorage` (`hathor/event/storage/rocksdb_storage.py:36`). It reuses the same low-level RocksDB wrapper as the ledger (`rocksdb_storage.py:20`) and carves out two column families (`rocksdb_storage.py:28`):

```
_CF_NAME_EVENT = b'event'          # id (8-byte big-endian) → serialized BaseEvent
_CF_NAME_META  = b'event-metadata' # bookkeeping: last group id, node state, ...
```

The key for an event is its id encoded as 8 big-endian bytes (`rocksdb_storage.py:85`), which means RocksDB stores events in id order on disk — so "iterate from id N" is a cheap seek-and-scan, not a full table walk. That is exactly what `iter_from_event` does (`rocksdb_storage.py:47`): seek to the requested id, then yield each subsequent record, deserializing it from JSON. `save_event` (`rocksdb_storage.py:77`) enforces the no-gaps invariant at write time — it raises `ValueError` if the incoming id is not exactly `last + 1` (`rocksdb_storage.py:81`), so a bug that skipped an id would fail loudly rather than corrupt the feed silently.

The Layer-2 invariant. Event ids are contiguous from 0, enforced both when building an event (`event_manager.py:315`) and when storing it (`rocksdb_storage.py:81`). A consumer that has seen id N has, by construction, seen every id $\leq N$. This is what makes "resume from N" correct.

Streaming to the world: the WebSocket layer

The live feed reaches external clients over a WebSocket⁴, handled by the `event/websocket/` sub-package. We cover it lightly here and defer the general WebSocket machinery to Chapter 36; what matters now is how it consumes the queue.

`EventWebsocketFactory` (`hathor/event/websocket/factory.py:31`) is a Twisted/Autobahn factory (Ch. 16's factory pattern): it builds one `EventWebsocketProtocol` per connecting client and keeps the set of live connections (`factory.py:59`). When `EventManager` calls `broadcast_event` (`factory.py:84`), the factory pushes the new event to every connection that is ready to receive it.

The "ready" qualifier is the interesting part, and it is what makes the feed reliable rather than fire-and-forget. Each client connection is a small state machine (`EventWebsocketProtocol`, `protocol.py:34`) implementing **acknowledged, windowed delivery**:

- The client opens the stream with a `START_STREAM` request (`request.py:22`) that says "I last acknowledged event id X; my window size is W." — i.e. "resume after X, and don't send me more than W unacknowledged events at a time."
- The node sends events strictly in id order, but only up to W ahead of the last id the client has acknowledged. The check is `can_receive_event` (`protocol.py:58`): the stream must be active, the event must be exactly the next one expected, and the number of in-flight (sent-but-unacked) events must be below the window.
- The client periodically sends `ACK` (`request.py:35`) to confirm it processed up to some id and to adjust its window. Each ack lets the node send more (`protocol.py:127`).

This back-pressure⁵ keeps a slow consumer from being flooded, and the resume-from-last-ack semantics, combined with the durable queue, are what let a consumer disconnect and reconnect without losing or duplicating events. If the client falls behind or reconnects, `send_next_event_to_connection` (`factory.py:105`) reads the missing events straight out of `EventStorage` and replays them — the same persisted records, no matter how long the client was away. That is the durable queue earning its keep.

The replayable backlog at startup

One more piece ties durability to the node's own history. When the event queue is first enabled on a node that already has a populated ledger, there is a backlog: thousands of vertices were accepted before the feed existed, and a faithful feed must include them. `handle_load_phase_vertices` (`event_manager.py:334`) handles exactly this. During the node's load phase it walks the existing vertices in topological order and manufactures a `NEW_VERTEX_ACCEPTED` event for each, batching them into storage in chunks of `N_LOAD_EVENTS_PER_BATCH` (= 10,000, `event_manager.py:38`). The decision to do this hinges on `_should_reload_events` (`event_manager.py:322`): if the previous node state was `None` or `LOAD`, the queue is (re)built from scratch; otherwise the node resumes from where it left off. This is why a consumer can trust the feed to be a complete history of the ledger, not merely "everything since the feature was switched on in this session."

30.6 How it plugs into the lifecycle

Both layers are created and started by machinery you've already met. The thread, in order:

Creation (the builder, Ch. 24). The `Builder` creates the single `PubSubManager` lazily via `_get_or_create_pubsub` (`hathor/builder/builder.py:444`), passing it the reactor; every subsystem that needs the bus is handed this same instance. The durable layer is assembled in `_get_or_create_event_manager` (`builder.py:538`): it builds the `EventRocksDBStorage` (`builder.py:535`), the `EventWebsocketFactory` (`builder.py:544`), and wires them — together with the shared `pubsub` — into the `EventManager` (`builder.py:550`). All of this lands on the `HathorManager` the builder produces.

Wiring (the manager, Ch. 29). The manager stores the bus as `self.pubsub` (`hathor/manager.py:189`) and hands it to subsystems that publish or subscribe — for example the wallet (`manager.py:223`) and, via `register_wallet`, a `NETWORK_NEW_TX_PROCESSING` subscription (`manager.py:261`).

Startup (the manager, Ch. 29). Inside `_initialize_components` the order is exact and worth noting (`manager.py:304`):

```
if self._enable_event_queue:
    self._event_manager.start(str(self.my_peer.id))    # Layer 2 subscribes to the bus

self.state = self.NodeState.INITIALIZING
self.pubsub.publish(HathorEvents.MANAGER_ON_START)  # first Layer-1 event
self._event_manager.load_started()                  # first Layer-2 event
```

The `EventManager` subscribes to the bus before the manager publishes `MANAGER_ON_START`, so no early event is missed. `EventManager.start` (`event_manager.py:92`) is where the durable layer comes alive: it decides whether to reload the queue or resume (`event_manager.py:101`), refuses to start if the previous run left an event group unclosed (`event_manager.py:110`, the crash-detection guard), subscribes to the bus (`event_manager.py:111`), and starts the WebSocket factory (`event_manager.py:114`). The load-phase backlog is generated later in initialization (`manager.py:476`), and `load_finished` is called when initialization completes (`manager.py:481`).

Steady state. From here, every accepted vertex (Ch. 32–33) flows through `vertex_handler` → `pubsub.publish(NETWORK_NEW_TX_ACCEPTED)` → bus drains on the next reactor tick → `EventManager._handle_hathor_event` → persisted as `NEW_VERTEX_ACCEPTED` → broadcast to WebSocket clients. The whole nervous system, end to end, on one event.

Shutdown. On stop, the manager publishes `MANAGER_ON_STOP` (`manager.py:355`) and stops the event manager (`manager.py:377`), which closes the WebSocket factory.

Recap

Layer	Purpose	Persistence	Audience	Built on
<code>PubSubManager</code> (<code>pubsub.py:188</code>)	decouple producers from consumers inside the process	none — ephemeral	the node's own components	callbacks (Ch 2) + observer pattern (Ch 3)
<code>EventManager</code> + event queue (<code>event_manager.py:59</code>)	a durable, ordered, replayable feed	RocksDB, gap-free <code>id</code> (<code>rocksdb_storage.py:36</code>)	external systems	the Layer-1 bus
<code>EventRocksDBStorage</code> (<code>rocksdb_storage.py:36</code>)	persist events as <code>id</code> → <code>BaseEvent</code>	RocksDB, two column families	(internal to Layer 2)	RocksDB (Ch 27)
<code>EventWebsocketFactory</code> / <code>Protocol</code> (<code>factory.py:31</code> , <code>protocol.py:34</code>)	stream the queue with ack + back-pressure	reads from storage on replay	external WebSocket clients	Autobahn/ Twisted (Ch 16, 36)

The two systems are one idea applied at two scales. `PubSubManager` is the observer pattern — a dictionary of callback lists — used so the node's components never have to know about each other; it is ephemeral by design, because every component that cares is alive at the same instant. `EventManager` is a subscriber to that bus that adds the three things outsiders need and insiders don't: a sequential `id`, durable RocksDB storage, and acknowledged WebSocket streaming. Keep the dividing line sharp — in-process and gone versus persisted and replayable — and the rest of the package is detail.

Events are how the node announces that something happened. The next chapter, **Chapter 31 (Verification)**, is about the very first stage a new vertex passes through before any such announcement is possible — the rule-checking that decides whether a vertex is even allowed to enter the ledger and, eventually, fire the `NEW_VERTEX_ACCEPTED` event you now know the full life of.

-
1. Decoupling means reducing how much two pieces of code need to know about each other. Tightly coupled code shares references and assumptions, so a change to one forces changes to the other. Loosely coupled code communicates through a narrow, stable interface — here, "publish an event" — so each side can change freely. ↩
 2. The event queue is Hathor's name for the durable, numbered log of events on disk. "Queue" here means an ordered sequence consumers read in order, not a transient in-memory buffer. It is the persistent half of the event system, off by default and enabled per node. ↩
 3. Pydantic is a Python library for declaring data models as classes with typed fields; it validates and coerces data at runtime and can serialize to/from JSON. Full treatment in Ch. 18. Here it gives `BaseEvent` its field types and the validator that keeps each event's data shape matching its type. ↩
 4. A WebSocket is a persistent, two-way connection between a client and a server over a single long-lived TCP link, unlike HTTP's request-then-close model. It lets the node push events to a client the moment they happen, rather than the client polling. Full treatment in Ch. 36. ↩
 5. Back-pressure is a flow-control technique where a fast producer is held back so a slower consumer is not overwhelmed. Here the client declares a window (how many unacknowledged events it can hold), and the node refuses to send beyond it until the client acknowledges, keeping memory bounded on both sides. ↩